4-bit Array Multiplier from RTL to GDSII (SKY130)

Digital VLSI Design: team assignment report

Name	Roll number
Sri Pranavadarsh Tadepalli	2024102028
Srikar Krishna Durbha	2024112010

Tools: IIC-OSIC-TOOLS Docker image in WSL 2 (Ubuntu 24.04). Icarus Verilog 14, GTKWave 4.0, Yosys 0.70, OpenSTA 3.1.0, LibreLane 3.1.0.dev3 (OpenROAD, Magic 8.3.685, KLayout 0.30.12, Netgen 1.5.324), SKY130A PDK, library `sky130_fd_sc_hd`, typical corner `tt_025C_1v80` for Parts B and C.

1 Part A: RTL design and verification

Design. `full_adder` uses two `assign` statements (`sum = a ^ b ^ cin`, `cout = ab + a·cin + b·cin`). `mult_array` registers `a` and `b` (8 flip-flops), forms the 16 partial products `pp_i[j] = a_r[j] & b_r[i]`, adds them in three rows of 4 `full_adder` instances (12 in total, built with `generate` loops and wired exactly as the Section 2 table), and registers the 8-bit result into `p` (8 flip-flops). All 16 flip-flops have an asynchronous active-low reset. The `*` operator is not used.

Testbench. 10 ns clock, `rst_n` low for the first 2 cycles, all 256 combinations applied on the falling edge, each checked two clock edges later against `a * b`, a line printed for each mismatch, one summary line, and `mult.vcd` written.

Check	Result
RTL simulation	`PASS: 256 vectors checked, 0 errors`
Error test (`sum = a ^ b`)	`FAIL: 256 vectors checked, 95 errors` (first: `a=3 b=3 p=5 expected=9`)

1.1 Worked example

Roll number used: Sri Pranavadarsh Tadepalli's, 2024102028 (last digit 8), so $A = 8 + 5 = \mathbf{13}$ and $B = \mathbf{7}$ ($A \neq 7$).

```
        1 1 0 1     A = 13
      x 0 1 1 1     B = 7
  ---------------
        1 1 0 1     pp0 = A AND b0 (b0 = 1)
      1 1 0 1       pp1 = A AND b1 (b1 = 1), shifted 1
    1 1 0 1         pp2 = A AND b2 (b2 = 1), shifted 2
  0 0 0 0           pp3 = A AND b3 (b3 = 0), shifted 3
  ---------------
0 1 0 1 1 0 1 1   P = 91
```

Through the array: Row 0 gives p[0] = 1 and r0 = {0, pp0[3:1]} = 0110. Row 1: 1101 + 0110 = 10011, so p[1] = 1, c1[4] = 1 and r1 = {c1[4], s1[3:1]} = 1001. Row 2: 1101 + 1001 = 10110, so p[2] = 0 and r2 = 1011. Row 3: 0000 + 1011 = 01011, so p[3] = 1 and p[7:4] = {c3[4], s3[3:1]} = 0101.

P = 0101 1011 = 91.

In the waveform, b changes to 7 at 4320 ns while a is already 13, the input registers capture them at clock edge 1 (4325 ns), and p = 91 appears at clock edge 2 (4335 ns): a latency of 2 clock cycles.

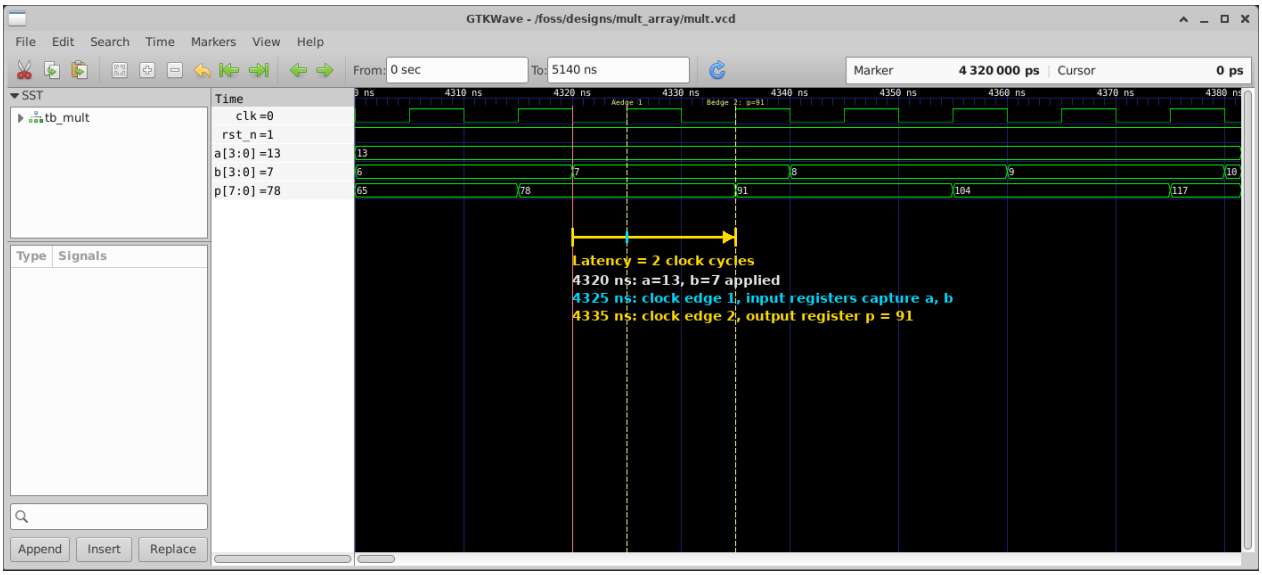


Figure 1: Waveform for a = 13, b = 7 with the 2-cycle latency marked

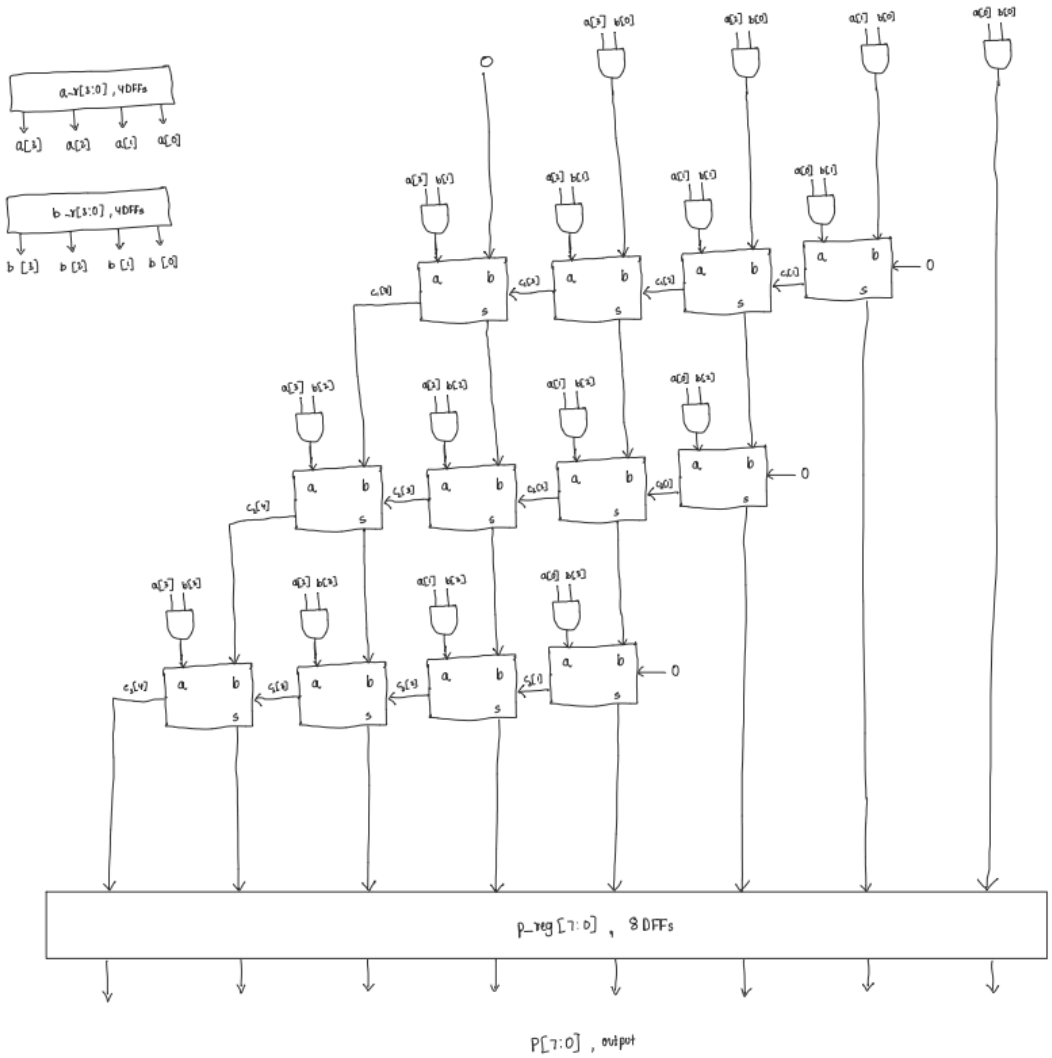


Figure 2: Block diagram: input registers, 16 AND gates, 12 full adders in three ripple-carry rows, output register

2 Part B: Logic synthesis with Yosys

Metric	Value
Total number of cells	61
Number of flip-flops	16 (`sky130_fd_sc_hd__dfrtp_1`)
Total cell area (μm^2)	756.976
Flip-flop area (μm^2)	$16 \times 25.024 = 400.384$
Flip-flop area (% of total)	52.89 %
The 3 most-used cell types and their counts	dfrtp_1 × 16, nand2_1 × 7, and2_0 × 4 and maj3_1 × 4 (tied)
Netlist simulation result	`PASS: 256 vectors checked, 0 errors`

Remaining cells: a21oi_1 × 3, xnor2_1 × 3, xor3_1 × 3, a22oi_1 × 2, clkinv_1 × 2, lpflow_isobufsrc_1 × 2, nand3_1 × 2, nand4_1 × 2, nor2_1 × 2, xor2_1 × 2, and one each of

a21boi_0, and3_1, and4_1, o2111ai_1, o21bai_1, o311ai_0, xnor3_1.

3 Part C: Static timing analysis with OpenSTA

Metric	Value
Worst slack at T = 10 ns (ns)	7.213
T_{min} (ns)	$10 - 7.213 = 2.787 \rightarrow 2.79$
F_{max} (MHz)	$1000/2.787 = 358.8$
Slack at T = T_{min} = 2.79 ns (ns)	0.003
Critical path start point	`_91_` = input register `a_r[1]` (dfrtp_1, clock pin)
Critical path end point	`_88_` = output register `p[6]` (dfrtp_1, D pin)
Number of cells on the critical path	8 logic cells (10 including the two flip-flops)
Latency (ns) = 2× T_{min}	5.57 (5.58 with T = 2.79 ns)
Throughput (million results per second)	358.8

Cells on the critical path (delays at T = 10 ns, arrival 2.652 ns, setup time 0.135 ns):

#	Instance	Cell	Delay (ns)	Role
0	`_91_`	dfrtp_1 (CLK→Q)	0.398	launch flip-flop `a_r[1]`
1	`_37_`	clkinv_1	0.073	partial-product logic
2	`_42_`	o311ai_0	0.356	partial product + row-1 adder (merged)
3	`_45_`	a21oi_1	0.153	row-1/row-2 adder logic (merged)
4	`_51_`	maj3_1	0.402	full-adder carry
5	`_58_`	xnor3_1	0.370	full-adder sum
6	`_61_`	maj3_1	0.368	full-adder carry
7	`_63_`	maj3_1	0.354	full-adder carry
8	`_64_`	xor3_1	0.178	full-adder sum (row 3, bit 3)
9	`_88_`	dfrtp_1 (D)		capture flip-flop `p[6]`

Five of the eight logic cells are full-adder cells (three maj3 carries and two xor3/xnor3 sums), compared with the estimate of about $2N = 8$ full-adder delays.

4 Part D: Physical design with LibreLane

`config.json` sets `DESIGN_NAME`, `VERILOG_FILES`, `CLOCK_PORT`, `FP_SIZING = "absolute"`, `DIE_AREA = [0, 0, 150, 150]` and `CLOCK_PERIOD`. The first period was $1.2 \times 2.787 = 3.34$ ns, rounded up to 3.5 ns. LibreLane signs off setup timing at all nine corners; the slow corner (ss, 100 °C, 1.60 V) failed, so the period was raised by 0.5 ns per run:

Run	CLOCK_PERIOD (ns)	Worst setup slack, all corners (ns)	Typical-corner slack (ns)	DRC	LVS
1	3.5	−2.546	+0.431	0	0
2	4.0	−2.027	+0.929	0	0
3	4.5	−1.527	+1.429	0	0
4	5.0	−1.027	+1.929	0	0
5	5.5	−0.527	+2.429	0	0
6	6.0	−0.037	+2.911	0	0
7	6.5	−0.205	+3.111	0	0
8	7.0	−0.116	+3.526	0	0
9	7.5	−0.138	+3.722	0	0
10	8.0	**+0.362**	+4.222	0	0

Metric	Value
CLOCK_PERIOD used (ns)	8.0
Number of runs needed	10
Core area (μm^2)	17,759.5 (die 22,500)
Cells after synthesis (from Part B)	61
Cells after place and route	1,884
Kinds of cells added	1,299 decap, 254 fill, 245 tap, 24 buffers (17 buf_2 timing repair, 3 clkbuf_16 clock tree, 4 clkdlybuf delay)
Wirelength (µm)	2,518
Setup slack (ns)	+0.362 (worst corner, max_ss_100C_1v60); +4.222 at typical
Post-route F_{max} (MHz)	$1000/(8.0 - 0.362) = 130.9$ (worst corner); $1000/(8.0 - 4.222) = 264.7$ (typical)
F_{max} from Part C (MHz)	358.8
DRC errors	0 (Magic 0, KLayout 0)
LVS errors	0

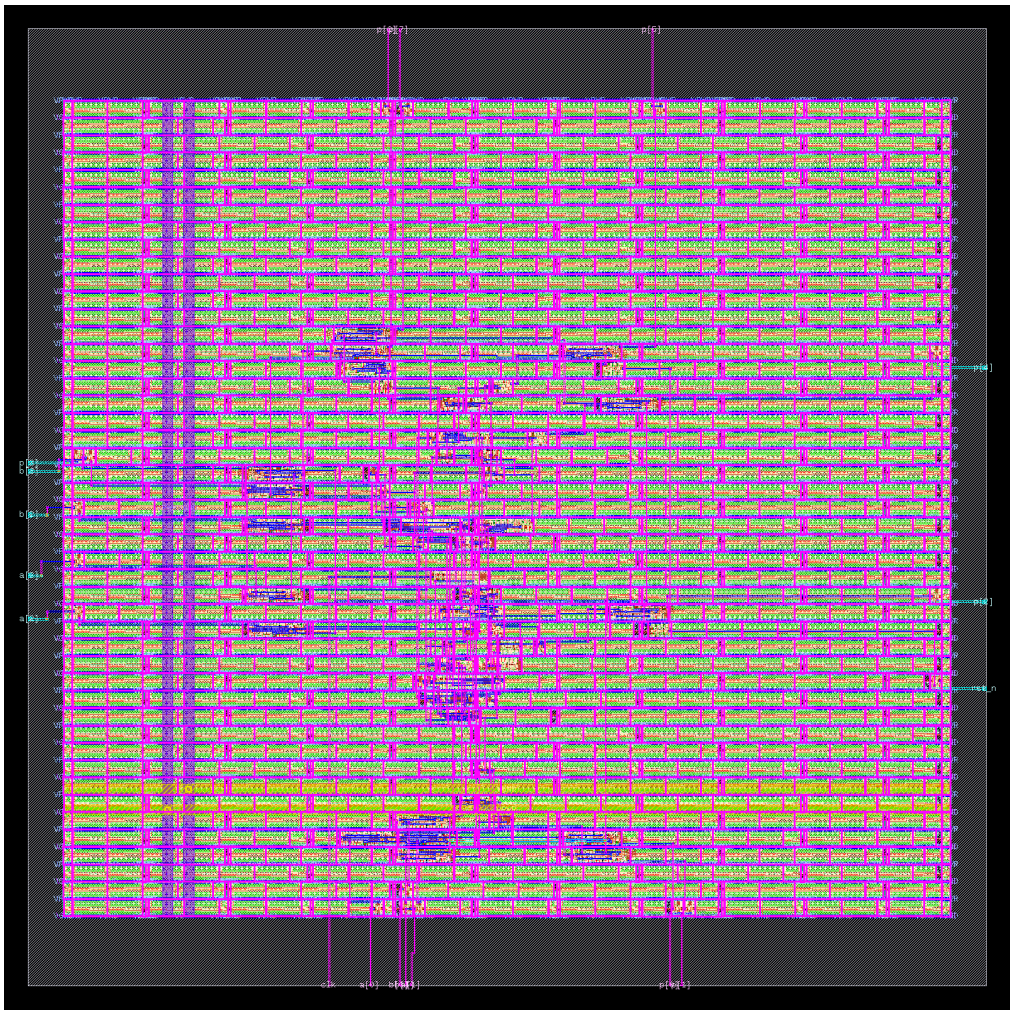


Figure 3: Final layout, whole 150 µm × 150 µm die

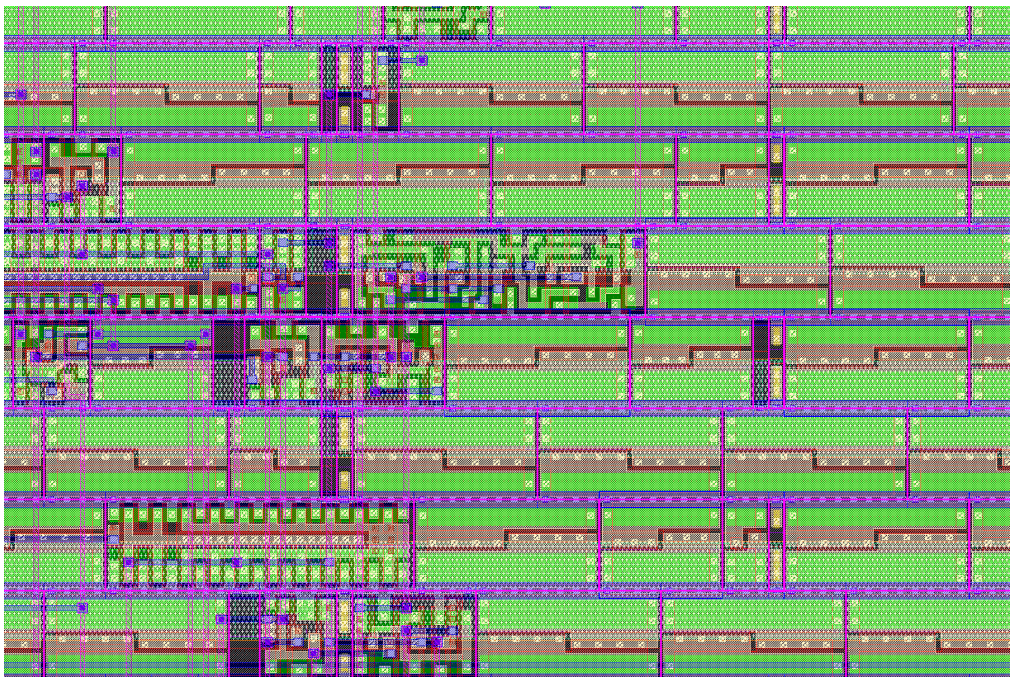


Figure 4: Zoomed view: standard-cell rows (filled with decap/fill cells), logic cells and met1/met2 wiring

5 Answers to the questions

1. Logic cells after synthesis. Synthesis produced 61 - 16 = 45 logic cells, not 16 + 12 = 28. This is because ABC does not keep the AND or full-adder boundaries, the design is flattened and remapped to whatever SKY130 cells give the smallest or fastest result. A full adder is usually two cells (maj3 for the carry and xor3 for the sum), so the 12 full adders alone account for about 24 cells. Some ANDs are also absorbed into compound gates (a21oi, a22oi, o311ai, nand3/4), while the adders which have a constant 0 input (c_i[0] = 0 and r0[3] = 0) are shrunk to half adders which are built from xor2 and and2. Inverters are added wherever a polarity change is required, and a few functions use cells such as lpflow_isobufsrc (which is an AND with one inverted input). The count is therefore a result of logic optimisation, not a one to one mapping.

2. Critical path. The critical path starts at input register a_r[1] and ends at output register p[6] (this is the sum of row 3, bit 3). It has 8 logic cells between the flip flops, 5 are full adder cells (maj3 carries and xor3/xnor3 sums). The 2N = 8 full-adder estimate assumes every stage is a full adder with the same delay but synthesis shortens this as each maj3 or xor3 computes a full adder's carry or sum in one cell, and the first adders (with constant 0 inputs) collapse into a few AOI/OAI gates, so the logic depth is about 5 full adder cells plus 3 simple gates. The path is still a carry ripple along a row that descends through the rows, which is the structure that the estimate describes.

3. Flip flop area. Flip-flops take 52.89 % of the cell area –> 400.384 of 756.976 µm^2. One dfrtp_1 25.0 µm^{2}which is 3–7 times larger than a simple gate (Ex: nand2_1 is 3.75 µm^2). This is because it contains two latches, a clock buffer and reset logic. The design has 16 flip flops but only 45 small gates. The number of registers grows according to $4N$ because of input and output bits, while the logic grows according to N^2, so for a small $N = 4$ multiplier the registers dominate but for larger N their share in total cell area would come down.

4. Post route $\mathbf{F_{max}}$. Post route F_{max} is 130.9 MHz at the worst corner and 264.7 MHz at the typical corner in comparison to 358.8 MHz in Part C. Part C used ideal wires an ideal clock and the typical corner only. After routing, the timing includes the resistance and capacitance of 2,518 µm of real wire, the clock-tree delay and skew, a 0.25 ns clock uncertainty, the delay of buffers added for slew and capacitance repair, and LibreLane signs off at the slow corner (ss, 100 °C, 1.60 V), where the cells are about 2.2× slower. The flow added tap cells (245) that connect the n-well and substrate to VDD and GND to prevent latch up, decap cells (1,299) that add capacitance between VDD and GND to steady the supply when many cells switch, fill cells (254) that close the gaps in the rows so wells, implants and power rails are continuous and density rules are met, and buffers (24): clock-tree buffers (clkbuf_16) that drive the clock to all 16 flip-flops with low skew, and timing/slew repair buffers that drive long or heavily loaded nets.

5. DRC and LVS. DRC or design rule check verifies that every shape in the layout obeys the foundry's geometric rules of minimum width, spacing, enclosure, overlap and density on each layer. LVS or layout versus schematic extracts the transistors and connections from the layout and checks that they form the same circuit as the netlist. Both need to pass, a DRC error means the chip may not be manufacturable or may have low yield due to shorts, opens, broken features and an LVS error means the chip would be manufactured correctly but would not be the circuit that was designed and verified. Our final layout has 0 DRC and 0 LVS errors.

6. Scaling to 8 bits. Full adders: $N(N-1) = 8 \times 7 = 56$ instead of $4 \times 3 = 12$ which is 4.7 times more, and $N^2 = 64$ AND gates instead of 16. Critical path: about $2N = 16$ full-adder delays instead of 8, so the logic delay roughly doubles. Using our Part C numbers, the flip-flop overhead is clock-to-Q + setup $\approx 0.398 + 0.135 = 0.53$ ns, so the 8 logic stages take about $2.787 - 0.53 = 2.25$ ns, about 0.28 ns per full-adder delay. For $N = 8$ –> $T_{min} \approx 0.53 + 16 \times 0.28 \approx 5.0$ ns, so F_{max} $\approx$ 200 MHz down from 359 MHz, while the logic area grows by about 4.7 times.

Work Distribution

The project and the report were completed as a team of two, with an equal division of work. Every part was written by one member and independently reviewed and tested by the other, so both members understand the complete design flow.

Part	**Sri Pranavadarsh Tadepalli**	**Srikar Krishna Durbha**
A: RTL and verification	`full_adder.v`, `mult_array.v`, block diagram,error-injection test	`tb_mult.v`, waveform analysis, hand-worked example
B: Synthesis	`synth.tcl`, netlist generation, Part B results table	Gate-level simulation, analysis of the cell and area report
C: Static timing analysis	`sta.tcl`, T_{min} and F_{max} calculation	Critical path analysis and Part C results table
D: Physical design	`config.json`, running LibreLane, collecting metrics , KLayout screenshots	comparison of synthesis and post-route results, Part D table
Report and README	Questions 4–6, final formatting and Git commits	Questions 1–3, README commands

GitHub Repository

The GitHub repo is this: https://github.com/Pranav-Adarsh/4-bit-Array-Multiplier-from-RTL-to-GDSII-